GROUP 3 DOCUTRUST

DocuTrust Project Guide

Secure Document Vault and Verification Platform - a beginner's companion to the hooks, architecture, database, server, tech stack and data API

> Stack: React Native 0.88 (Expo SDK 58, JavaScript) + Pure Java 21 (Maven) + PostgreSQL (Supabase)
>
> Scope: 46 screens - 234 numbered features - 3 roles - 74 API paths - 15 tables - 12 migrations
>
> Team: 6 students - 46 screens - 234 numbered features - expo-doctor 20/20 on SDK 58
>
> Chapter 14: the eight core concepts explained for absolute beginners (like I'm 5)
>
> Chapter 15: the six defence domains in plain words - five reusable analogies and glossary picks per student
>
> Proof style: live demos on the emulator and via manual API calls, greppable finding IDs, one-command Maven build

Table of Contents

1. Project Hook — What Is DocuTrust?.....
   1.1 The problem story.....
   1.2 What makes it genuinely hard.....
2. The Tech Stack in Plain Words.....
   2.1 React Native and Expo — the mobile client.....
   2.2 Pure Java 21 with Maven — the server.....
   2.3 PostgreSQL on Supabase — the database.....
3. Project and Folder Architecture.....
   3.1 The repository tree, annotated.....
   3.2 The life of a request.....
4. The Database Explained.....
   4.1 The tables and what each one remembers.....
   4.2 Constraints: rules the engine enforces.....
   4.3 Transactions: all or nothing.....
   4.4 Triggers and views: logic that cannot be forgotten.....
5. The Server Explained (Pure Java REST API).....
   5.1 The engine and the router.....
   5.2 Middleware: the checkpoint chain.....
   5.3 Authentication: hand-rolled JWT with rotation.....
   5.4 The FileVault: AES-256-GCM at rest.....
   5.5 Background workers: the scheduler and the seeder.....
6. Data Consumption API — How the App Talks to the Server.....
   6.1 Envelopes, errors and pagination.....
   6.2 The endpoint map by module.....
   6.3 How the client consumes it.....
   6.4 Sharing and verification flows from the API's point of view.....
   6.5 SEO metadata.....

7. MVP Pattern and Frontend Microservices.....
7.1 MVP in one sentence each.....
7.2 Frontend microservices.....
8. The Six Hardest Challenges, Solved Twice.....
8.1 Access control.....
8.2 Private file delivery.....
8.3 Expiring links and revocation.....
8.4 Audit trails.....
8.5 Document-version integrity.....
8.6 The administrative review process.....
8.7 The OTP security layer (post-audit addition).....
9. How Correctness Is Shown.....
10. How to Run Everything.....
10.1 The database (Supabase).....
10.2 The server.....
10.3 The mobile app.....
11. The Three-Week Roadmap.....
12. The Edge-Case Demonstrations.....
13. Features, Roles and Screens at a Glance.....
14. Explain It Like I'm 5 — The Eight Core Concepts.....
14.1 React Hooks — the sticky notes that remember things (rehearse: Student 4).....
14.2 The Overall Architecture — the customer, the waiter, the kitchen (rehearse: Student 1).....
14.3 The Frontend Folders — it's like a smartphone (rehearse: Student 4).....
14.4 The Backend Folders — a well-organized restaurant (rehearse: Student 1).....
14.5 The Database — a highly organized filing cabinet (rehearse: Student 1).....
14.6 The Server — a 24/7 receptionist answering phones (rehearse: Student 6).....
14.7 The Tech Stack — what we chose and why (rehearse: Student 6).....
14.8 The Data Consumption API — the standardized order form (rehearse: Student 3).....
15. The Six Defence Domains in Plain Words.....
Student 1 — Infrastructure & Database.....

Student 2 — Auth & OTP Security.....
Student 3 — Business Logic (Documents, Sharing, Verification).....
Student 4 — Mobile App.....
Student 5 — The 8-Pass Security Audit.....
Student 6 — Deployment & DevOps.....

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

1. Project Hook — What Is DocuTrust?

DocuTrust is a **secure document vault and verification platform** built for individuals and organisations. Think of it as a bank vault for your important documents: passports, national IDs, driver's licences, contracts, certificates, invoices, medical records and academic papers all live in one encrypted place. But DocuTrust does three things an ordinary cloud folder cannot do. First, it knows when your documents expire — a passport expires in six months and DocuTrust warns you before it does. Second, it lets you share a document safely, through links that expire, can be revoked instantly, can limit how many times they are viewed, can demand a passcode — and can even be locked until a specific person proves control of their email inbox with a one-time code. Third, it lets a third party — a bank, an employer, a university — verify that your document is genuine without ever downloading the file itself.

The project was written by a team of six students as a full production-style system: a mobile application built with **React Native, Expo and JavaScript**, a backend built with **pure Java and Maven** (deliberately without the Spring framework and without JUnit), and a relational database running **PostgreSQL on Supabase**. The application has 46 screens, 234 individually numbered features (216 core plus 18 added by the post-audit security overhaul), **74 API routes**, **15 database tables** across 11 migrations, three user roles (admin, org_manager and member), a runnable one-command build, documented API contracts, architecture diagrams and an entity-relationship diagram. Every one of those numbers is listed, itemised and traceable in the repository, because the project is designed to survive a hard oral defence.

1.1 The problem story

Almost everyone has lost time to a document problem. A visa application fails because an ID card expired last week and nobody noticed. A landlord asks for proof of employment and the contract is buried in an email attachment from two years ago. A recruiter cannot tell whether the certificate a candidate emailed is genuine or a Photoshopped fake. Organisations suffer the same problems at scale: HR departments chase employees for compliance certificates, and sensitive files circulate by email with no control over who still has a copy. These failures share three root causes: documents are stored without any expiry awareness, they are shared without any access control, and their authenticity cannot be proven without a trusted authority.

DocuTrust attacks each cause directly. Expiry awareness is built into the data model: every document row carries an `expires_at` date and a per-document reminder window, and a background scheduler transitions documents through an active, expiring and expired lifecycle while generating notifications. Safe sharing is implemented through a purpose-built

share engine with expiring links, view limits, passcodes and instant revocation, all enforced by one atomic SQL statement. Provable authenticity is delivered by verification links that expose a cryptographic proof — the document's SHA-256 hash, issue date and status — without releasing the document bytes.

1.2 What makes it genuinely hard

The brief named six hard challenges, and the whole architecture is organised around them. **Access control**: every request is authenticated with a JSON Web Token and every role and ownership rule is enforced in server code and SQL, never in the app. **Private file delivery**: uploaded files are encrypted with AES-256-GCM before they touch the disk and are streamed only through authorised endpoints — there is no public URL to a document, ever. **Expiring share links**: every open re-checks expiry, revocation and view count inside one atomic UPDATE, so a link cannot be used after death, even by two people at the same moment. **Revocation**: revoking a link takes effect immediately because the check happens on every request, not at link-creation time. **Audit trails**: database triggers make the history and audit tables append-only — no code path, and no DBA shortcut, can silently rewrite the past. **Document-version integrity**: every version pins a SHA-256 checksum that is recomputed on download, which is what makes the tamper-detection demo possible.

Challenge	Where it is solved	Chapter
Access control	JWT middleware + RoleGuard + SQL ownership scoping	5
Private file delivery	AES-256-GCM vault + authorised streaming endpoints	5, 6
Expiring share links	Atomic guard UPDATE in ShareService	6
Revocation	revoked_at checked on every request	6
Audit trails	Append-only triggers + audit_logs table	4
Version integrity	SHA-256 per version, recomputed on download	4, 5

Table 1. The six hardest challenges and where they live in the codebase.

2. The Tech Stack in Plain Words

A technology stack is just the set of tools a system is built with, layer by layer. DocuTrust has three layers: the app people touch (the mobile client), the program that thinks (the REST server), and the place where facts are kept (the database). This chapter explains each tool, why it was chosen and what job it does, in plain language.

2.1 React Native and Expo — the mobile client

React Native is a framework for building real mobile applications using JavaScript and React. Instead of writing the iOS app in Swift and the Android app in Kotlin separately, React Native lets us write one codebase whose components compile down to genuine native user-interface elements on both platforms. Expo is a toolchain that sits on top of React Native and removes the most painful setup work: with Expo we do not need Xcode or Android Studio configured to build and run the app, we get a development app on our phones by scanning a QR code, and project configuration lives in a single file called `app.json`. The Expo document-picker and file-system packages handle the mechanics of choosing a file on the phone and handing its bytes to our HTTP client. DocuTrust targets **Expo SDK 58** with **React Native 0.88** - the newest SDK generation, which brings the new React Native architecture and iOS 27 support - and two Expo commands keep the stack healthy: `npx expo install --fix` aligns every dependency to the version the SDK certifies, and `npx expo-doctor` inspects the project like a mechanic and currently reports 20/20 checks clean.

The client is written entirely in JavaScript (no TypeScript) as the brief requires, and it is organised as nine independent **frontend microservices** — auth, documents, folders, sharing, verification, notifications, admin, org and profile. Each microservice owns its own API calls, its own state model and its own presenters, and registers its screens in a central registry. Chapter 7 explains this pattern and why it is more than a folder-organisation trick.

2.2 Pure Java 21 with Maven — the server

The backend is a plain Java program started from a `main()` method, using the JDK's built-in HTTP server class `com.sun.net.httpserver.HttpServer`. Maven is the build tool: its `pom.xml` file compiles the code and packages it into a jar. The runtime dependencies are deliberately few and surgical: the PostgreSQL JDBC driver plus four infrastructure libraries — **HikariCP** for connection pooling, **Bucket4j** for rate limiting, **Caffeine** for caching and **Dropwizard Metrics** for request telemetry — each one swapping the internals of a hand-rolled component without changing the code that calls it. No Spring, no JUnit — both are explicitly forbidden by the brief.

Spring was forbidden for a good pedagogical reason: frameworks hide decisions, and this course is about making decisions. Without Spring we had to write our own router that maps URL patterns like `/api/documents/:id` to handler methods, our own middleware chain for CORS, logging, authentication, role checks, rate limiting and error handling, and our own JSON parser and serialiser. About two thousand lines of the codebase are the kind of plumbing that Spring would normally provide, and every line of it is examinable. The same logic applies to tooling: everything the build needs is either in the JDK or pinned in the pom, so `mvn clean package` is the one command that turns a clean clone into the runnable jar.

2.3 PostgreSQL on Supabase — the database

PostgreSQL is a relational database: data lives in tables with fixed columns, rows reference each other through foreign keys, and the engine enforces every rule we declare. Supabase is a hosted Postgres service — the same database engine, managed in the cloud, with a connection string we paste into our environment file. We chose Postgres over a document store such as MongoDB because DocuTrust's data is intensely relational: a share always belongs to exactly one document and one owner, a version always belongs to one document, and the integrity rules (a share must have a user or a token, a sensitive document must enter review) are best expressed as database constraints rather than application code that could be bypassed. The schema is created and evolved by eleven numbered SQL migration files, applied in order by a small Java migration runner that records what it has applied in its own table.

Layer	Technology	Why it was chosen
Mobile app	React Native 0.88 + Expo SDK 58, JavaScript	One codebase for Android and iOS; Expo removes native build pain; expo-doctor 20/20
App architecture	MVP pattern + frontend microservices	Testable screens; independent feature modules
Server	Pure Java 21 + Maven	Zero-framework transparency; every decision visible
Server HTTP	com.sun.net.httpserver (JDK built-in)	Production-grade HTTP engine included in the JDK
Database	PostgreSQL via Supabase	Constraints, transactions and triggers are enforced in the engine
Crypto	PBKDF2, HMAC-SHA256, AES-256-GCM (JDK)	Standard, audited algorithms hand-wired, no dependencies

Table 2. The stack at a glance.

3. Project and Folder Architecture

The repository has four top-level directories, and the split is deliberate: `mobile/` is the client, `server/` is the backend, `database/` holds the SQL migrations that define the schema, and `docs/` holds the contracts and diagrams that the other three must obey. Keeping contracts in the repo means a new team member can read the API contract and the migration files and know exactly what the system does before reading a single line of Java or JavaScript.

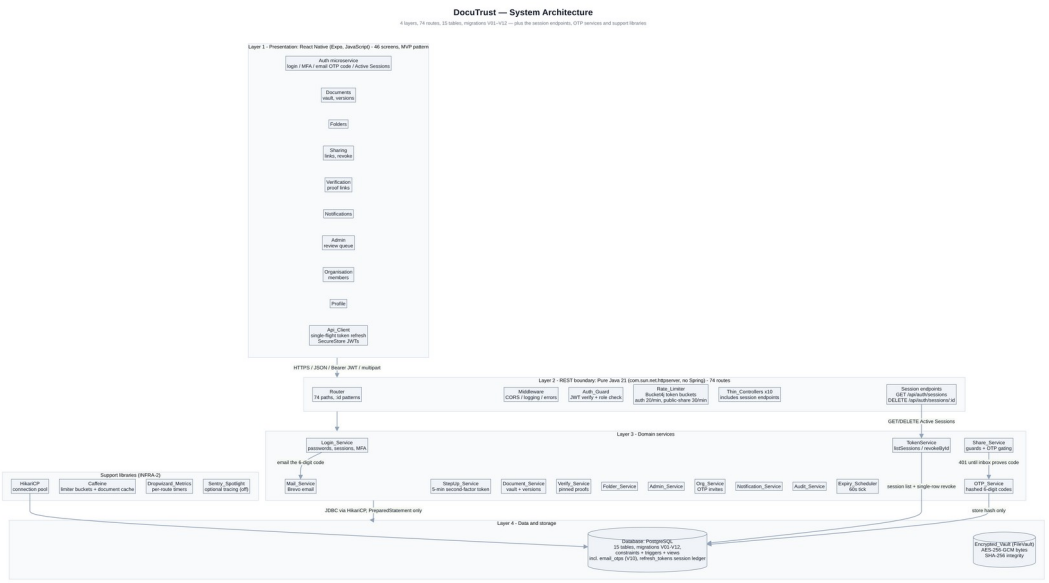


Figure 1. DocuTrust system architecture: four layers, from the React Native client to the encrypted vault.

3.1 The repository tree, annotated

The tree below is abridged but honest — every directory shown exists in the repository. Under the server, the package layout follows a **modular monolith**: the HTTP layer, the business services and the database access code are strictly separated packages, so the system behaves like well-factored layers even though it deploys as one process. The mobile side mirrors this with its nine microservice modules.

Path	What lives there
database/migrations/V01–V11.sql	Schema evolution: extensions, 15 tables, constraints, indexes, triggers, views (V10 adds email_otps, V11 the OTP-gated share columns)
server/src/main/java/com/docutrust/http/	Router, request context (Ctx), middleware chain, Handler interface
server/src/main/java/com/docutrust/auth/	JWT service, PBKDF2 passwords, refresh-token rotation, TOTP MFA, Bucket4j rate limiter
server/src/main/java/com/docutrust/service/	AuthService, DocumentService, ShareService, VerifyService, AdminService, OrgService, OtpService, MailService, StepUpService, ExpiryScheduler, Bootstrap
server/src/main/java/com/docutrust/controller/	Ten thin controllers that parse requests and call services
server/src/main/java/com/docutrust/db/	Connection pool, migration runner, transaction helper

Path	What lives there
server/src/main/java/com/docutrust/vault/	FileVault: AES-256-GCM encryption and streaming decryption
mobile/src/core/	API client with silent refresh, auth store, theme, utilities
mobile/src/mvp/	BasePresenter state machine + usePresenter React hook
mobile/src/microservices/	9 modules, each with api.js, model.js, presenters.js
mobile/src/screens/	45 screen views + the SCREEN_REGISTRY

Table 3. Annotated repository map.

3.2 The life of a request

Follow one request end to end and the architecture explains itself. The user taps Save on the Upload screen. The screen's presenter validates the form, asks the document-picker for the file, and calls the ApiClient, which attaches the stored access token. The request crosses the internet as multipart form data and lands in the Java Router, which matches the path `POST /api/documents`, runs the middleware chain — CORS, request logging, JWT authentication, role check, rate limit — and hands a clean request context to DocumentController. The controller is deliberately boring: it parses fields and delegates to DocumentService. The service does the real work inside one database transaction: validate, compute the file's SHA-256 while streaming it to the vault, encrypt the bytes with AES-256-GCM, insert the document row and its first version row, write a history row and — if the document type is sensitive — open a row in the admin review queue. Only then does the transaction commit, and a JSON envelope flows back: `{ ok: true, data: {...} }`. Every request in the system follows this same choreography, which is exactly why it is easy to defend in an exam.

4. The Database Explained

The database is the only component whose memory is permanent, so it is also the component that enforces the rules that must never be bent. DocuTrust's schema has fifteen tables, eleven migrations, dozens of constraints and six triggers. This chapter tours the schema table by table and then explains the three database features that make the security claims honest: constraints, transactions and triggers.

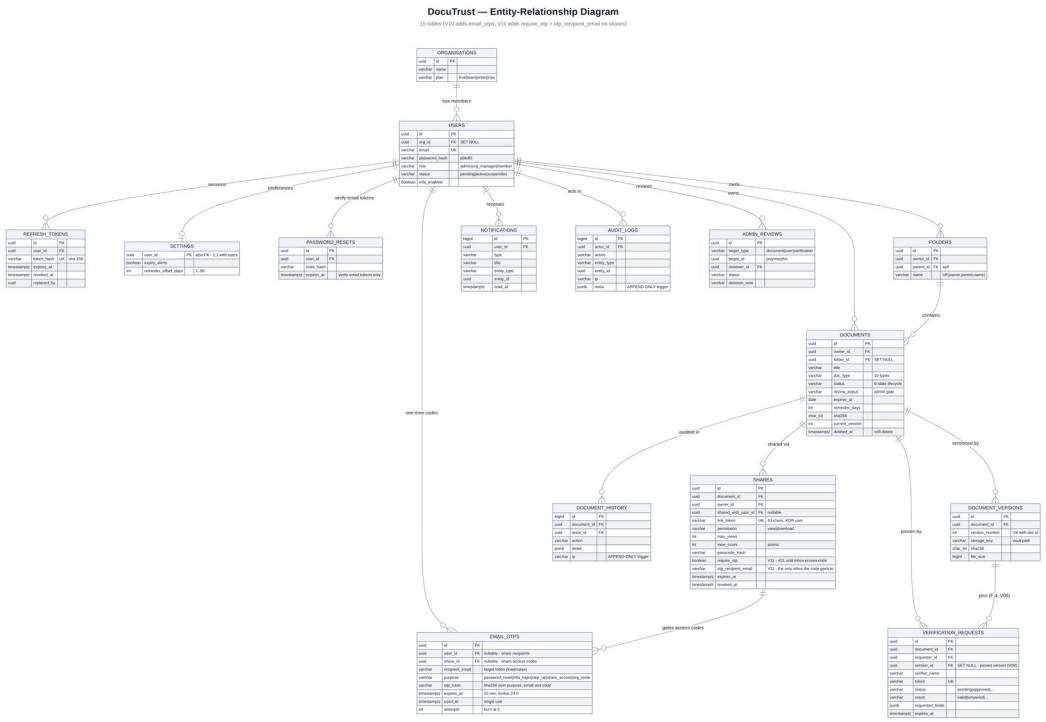


Figure 2. Entity-relationship diagram. Solid lines are the key foreign keys; the panel below lists every relationship.

4.1 The tables and what each one remembers

Identity is handled by four tables. `organisations` holds workspace accounts; `users` holds every person, with a role column constrained by a CHECK to exactly three values — admin, org_manager or member — and a status column that lets an admin suspend an account. `refresh_tokens` stores only the SHA-256 hash of each refresh token, its expiry and whether it has been rotated or revoked; the raw token is never stored, so a database leak cannot produce usable tokens. `password_resets` stores single-use, fifteen-minute reset codes, also hashed. The `settings` table is a one-to-one extension of users holding notification preferences.

The vault core is three tables. `folders` organises documents; a UNIQUE constraint on owner, parent and name forbids duplicate folder names, and a self-referencing foreign key allows nesting. `documents` is the heart of the system: it stores metadata only — title, one of ten document types, a six-state status lifecycle, an expiry date, a reminder window, the current version number and the mirrored SHA-256 of the newest version. The bytes themselves never sit in this table; they live encrypted in the vault directory on the server's disk. `document_versions` holds one row per upload: the version number, the encrypted storage key, the file's SHA-256, its size and mime type. A UNIQUE constraint on document and version number is what turns a concurrent-upload race into a clean error instead of

corruption. `document_history` is the per-document audit trail — actor, action and a JSON detail column — protected by a trigger that rejects any UPDATE or DELETE.

Sharing and verification are two tables with strong shapes. `shares` represents either a direct share to another user or a public link; a CHECK constraint insists that one of the two targets exists, another CHECK validates the 43-character token shape, and the row carries permission level, optional maximum view count, hashed passcode, mandatory expiry and a revocation timestamp. Migration V11 adds two more columns: `require_otp` flips the link into an OTP-gated state, and `otp_recipient_email` pins the one inbox whose code can unlock it. `verification_requests` stores a proof link: verifier name and organisation, purpose, a unique token, a two-state approval flow and, once checked, a result value such as valid, tampered, expired or revoked. Finally, the platform tables: `notifications` (the in-app inbox), `admin_reviews` (the administrative review queue for sensitive uploads and flagged accounts), and `audit_logs` (the platform-wide, append-only record of security-relevant events).

The newest table, `email_otps` (migration V10), is the engine of the whole OTP security layer and it is worth a slow read. Every one-time code the system issues — password resets, the email MFA fallback at login, step-up confirmations, share-access codes and organisation invites — lands here as one row: the recipient's address, a purpose tag, an expiry (ten minutes for personal codes, twenty-four hours for invites), an attempts counter and a used-at timestamp. The code itself is never stored: only a SHA-256 digest computed over the purpose, the address and the code, so a digest leaked from the database is useless for any other purpose or person. Issuing a code retires the previous unused one for the same target, and verification burns the row after five wrong attempts, all inside one locked transaction.

4.2 Constraints: rules the engine enforces

The schema pushes every invariant it can into the database itself, which is a defence-in-depth argument worth making out loud in a defence: application code can have bugs, but a declared constraint is enforced by the engine on every statement, including manual ones typed by an administrator. CHECK constraints pin the allowed values of roles, statuses, document types, share permissions and reminder windows. Foreign keys define what happens when a row dies: deleting a user cascades to their documents and tokens, but deleting a folder only detaches its documents by setting folder_id to NULL, so no document is ever lost by folder housekeeping. Uniqueness rules stop duplicate emails, duplicate folder names and, crucially, two concurrent uploads claiming the same version number. One business rule is so important it became a CHECK with a subquery-free formulation: a document of type medical or financial can never be active unless its review status is submitted — sensitive uploads must pass through the administrative review process.

4.3 Transactions: all or nothing

A transaction is the database's promise that a group of statements either all succeed or all vanish. DocuTrust wraps every multi-step write in a transaction through a small Tx helper that begins the transaction, runs the work, commits on success and rolls back on any exception. The upload path is the flagship example: insert the document, insert version one, write the history row, maybe open the review row — if the process dies half-way through, the vault would hold an orphaned encrypted file, so the service deletes the file and rolls back together. The share-open path uses the same discipline for a different reason: incrementing the view counter must be atomic with respect to the guards, which is why the service performs a single conditional UPDATE whose WHERE clause encodes not-revoked, not-expired and under-max-views, then checks whether exactly one row changed. Two people clicking an expiring link at the same second cannot both push an exhausted link over its limit.

4.4 Triggers and views: logic that cannot be forgotten

Three triggers give the schema its append-only spine and its audit honesty. First, trigger functions stamp updated_at on users and documents automatically. Second, an append-only guard raises an exception on any UPDATE or DELETE against document_history and audit_logs — the team demonstrated this live by typing an UPDATE in the SQL editor and showing the database refuse it. Third, an after-insert trigger on documents writes the created history row itself, and an after-update trigger writes status and version changes; even a code path that forgot to log cannot actually forget. A fourth trigger keeps the document row's sha256, size and current_version synchronised with the newest version row, eliminating a whole class of desynchronisation bugs. On top of the tables sit two SQL views: `v_expiring_documents`, which computes days-left for the Expiring screen, and `v_admin_stats`, which feeds the admin dashboard counters. Views keep that logic in one tested place instead of scattering it through controllers.

5. The Server Explained (Pure Java REST API)

The server is where every security claim is either kept or broken, so this chapter walks through its five moving parts: the HTTP engine and router, the middleware chain, the authentication design, the encryption vault, and the background workers. The running theme is that the server trusts nothing — not the client, not the network, not even the rest of itself beyond its own layer boundaries.

5.1 The engine and the router

Java's standard library ships with a complete HTTP server implementation, `com.sun.net.httpserver.HttpServer`, which is what Spring Boot uses underneath

anyway. Main.java creates it on port 8080, registers seventy-four route paths with the router, applies the middleware chain, runs the database migrations, seeds demo data on first boot and starts a background scheduler. The Router is a hand-written class that turns patterns like `/api/documents/:id/versions/:v/download` into segments and matches incoming paths against them, extracting the id and v parameters and parsing the query string. A route registration reads like a table row: method, pattern, and a handler wrapped in whichever guards it needs — public, authenticated, or role-restricted. Handlers receive a Ctx object that wraps the raw exchange with helpers for reading JSON bodies, parsing multipart forms, reading path and query parameters and streaming binary responses.

5.2 Middleware: the checkpoint chain

Every request passes through the same chain before any business code runs. CORS middleware answers browser preflights and stamps allowed origins so the Expo web build can talk to the API. Logging middleware writes a line per request with status and duration. The authentication middleware extracts the Bearer token, verifies the JWT's HMAC-SHA256 signature and expiry with a constant-time comparison, loads the user's claims into the context and converts problems into precise errors: a missing or malformed token is UNAUTHORIZED, a well-formed but stale token is TOKEN_EXPIRED, which the mobile client treats as its cue to refresh. The role guard is a one-liner per route — requireRole(admin) — that compares the token's role claim and writes an admin_access_denied audit row on every refusal, which is the evidence trail for the role-escalation demo. Finally the rate limiter guards the door: Bucket4j token buckets, one per scope-and-IP pair and held in a Caffeine cache, with per-scope budgets — twenty authentication attempts per minute, thirty for the public share endpoints, twenty for verification lookups. A bucket that runs dry answers 429 RATE_LIMITED; token buckets also smooth bursts instead of allowing a full window's worth of requests in one stampede.

5.3 Authentication: hand-rolled JWT with rotation

A JSON Web Token is three Base64-URL strings joined by dots: a header naming the algorithm, a payload of claims, and an HMAC-SHA256 signature of the first two parts made with a server secret. DocuTrust builds and verifies them with the JDK's Mac class — roughly forty lines of code, all readable in an exam. The payload carries the user id, role, organisation id, issued-at, a fifteen-minute expiry and a unique token id. Refresh tokens are a different animal: 43-character cryptographically random strings, stored only as SHA-256 hashes with a seven-day expiry. On refresh, the server verifies the hash, revokes the old row, links it to its replacement through replaced_by and issues a new pair. If an already-rotated token is ever presented again — the signature of a stolen token — the service returns TOKEN_REUSE and revokes the user's entire token family, forcing a fresh login. Passwords are hashed with

PBKDF2-HMAC-SHA256 at 120,000 iterations with a per-user random salt, stored in the self-describing format pbkdf2$120000$salt$hash and verified in constant time. TOTP multi-factor authentication is implemented to RFC 6238 with a Base32 secret and a plus-or-minus one time-step window, so the MFA demo works with any standard authenticator app.

On top of that sits the OTP layer, added in the post-audit security overhaul, and its core idea is simple: a credential that only ever exists in your inbox. When a flow needs proof — a password reset, a login without the authenticator app, a destructive action like permanently deleting a document, opening an OTP-gated share link, or claiming an organisation invite — the server generates a six-digit code with SecureRandom, stores only the purpose-salted SHA-256 digest, and emails the code through Brevo's transactional API. The API response never carries the code (in development mode it is echoed in an X-Dev-OTP header so demos work without mail traffic). The email MFA fallback answers login attempts with HTTP 440 and a challenge token; step-up verification exchanges a code for a five-minute, single-purpose JWT that destructive endpoints demand through the X-Step-Up-Token header; and the old patterns of returning reset codes or temporary passwords in response bodies were removed outright — the inbox is now the only place a credential exists.

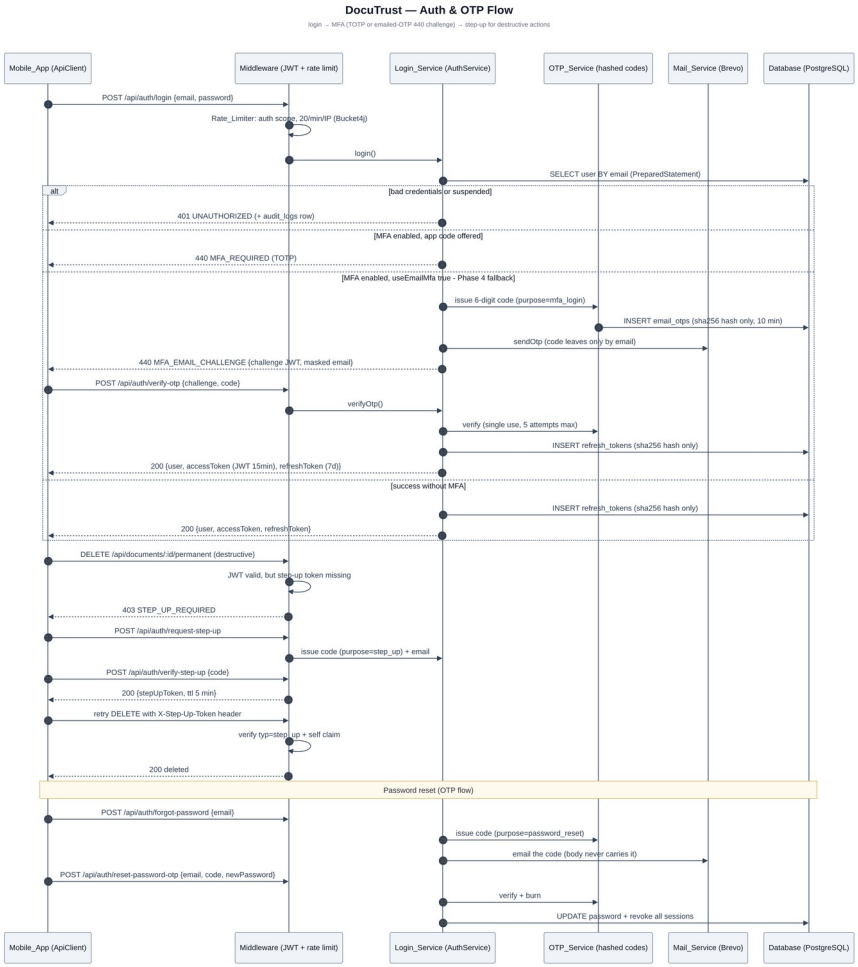


Figure 3. The authentication lifecycle: sign-in, per-request guards, silent refresh rotation and reuse detection.

5.4 The FileVault: AES-256-GCM at rest

Uploaded files never reach the disk in plaintext. DocumentService streams the upload once, computing the SHA-256 digest as bytes flow past, then hands the stream to FileVault, which generates a fresh 12-byte initialisation vector, encrypts with AES in Galois/Counter Mode using a 256-bit key from the environment, and writes IV-plus-ciphertext to a vault directory as a .bin object. GCM is an authenticated mode: it appends a tag that makes any later modification of the stored bytes detectable, which is what powers the tamper demo. Downloads go the other way: the service checks authorisation in SQL, asks FileVault to decrypt and stream, and recomputes the digest while streaming — if the recomputed digest disagrees with the stored one, the response aborts with an integrity error and an audit row is written. There is deliberately no static file server, no signed S3-style URL and no way to reach a vault object except through an authenticated endpoint that has just passed an ownership check.

5.5 Background workers: the scheduler and the seeder

Two background components keep the system alive between requests. The ExpiryScheduler ticks every sixty seconds: it moves active documents whose expiry falls inside their reminder window into the expiring state, moves expiring documents past their expiry date into expired, raises exactly one notification per document per day so users are warned without being spammed, and retires verification requests past their own expiry. The Bootstrap seeder runs only when the users table is empty: it creates a demo organisation, six accounts matching the six students, two folders, five documents in different lifecycle states — including one expiring in three days and one medical document parked in review — one active share and one approved verification request. The seeder builds a minimal valid PDF in code and pushes it through the same encryption path as a real upload, so the demo data is byte-for-byte representative of production data.

6. Data Consumption API — How the App Talks to the Server

REST is a style of API where the client and server exchange JSON over HTTP: the URL names a resource, the HTTP verb names the action, and the status code names the outcome. DocuTrust exposes 74 route paths, all documented in the repository's docs/API_CONTRACTS.md — the same file both teams coded against, which is why the contract document matters as much as the code. This chapter explains the conventions every endpoint shares and how the mobile client consumes them.

6.1 Envelopes, errors and pagination

Every response is wrapped in a consistent envelope so the client never parses special cases. Success returns `{ ok: true, data: ..., meta?: {...} }`; failure returns `{ ok: false, error: { code, message, field? } }` with an appropriate HTTP status. Error codes are a small controlled vocabulary — VALIDATION_ERROR, UNAUTHORIZED, TOKEN_EXPIRED, FORBIDDEN, NOT_FOUND, CONFLICT, SHARE_EXPIRED, SHARE_REVOKED, RATE_LIMITED and friends — and the mobile ApiClient turns each one into the right behaviour: TOKEN_EXPIRED triggers a silent refresh and retry, TOKEN_REUSE forces a logout, SHARE_PASSCODE_REQUIRED surfaces a passcode field. List endpoints share one pagination convention: page starts at one, limit defaults to twenty and never exceeds one hundred, and the response's meta block returns total and pages so the UI can render Load-more affordances honestly. Search uses the q-style parameter against PostgreSQL trigram indexes, so contains-style searches on titles stay fast as the vault grows.

6.2 The endpoint map by module

Module	Representative endpoints	Guard
Auth	POST /api/auth/register · /login · /verify-otp · /refresh · /logout · /forgot-password · /reset-password-otp · /accept-invite · /request-step-up · /verify-step-up · GET /api/auth/me · MFA setup/enable/disable	public + Bearer
Documents	GET/POST /api/documents · GET/PATCH/DELETE /api/documents/:id · /restore · /permanent · POST /:id/versions · GET /:id/versions/:v/download · /history · GET /api/documents/expiring	Bearer
Folders	GET/POST /api/folders · PATCH/DELETE /api/folders/:id · GET /:id/documents	Bearer
Sharing	GET/POST /api/shares · /:id/revoke · DELETE /:id · GET /:id/stats · GET /incoming	Bearer
Public	GET /api/public/shares/:token · POST /:token/open · /:token/request-otp · /:token/open-with-otp · GET /download/:ticket · GET/POST /api/public/verify/:token	public, rate-limited
Verification	POST/GET /api/verification-requests · /:id/approve · /:id/reject	Bearer
Notifications	GET /api/notifications · POST /:id/read · /read-all · GET/PUT /api/settings/notifications	Bearer
Admin	GET /api/admin/stats · /review-queue · POST /reviews/:id/decision · GET /users · PATCH /users/:id/status · GET /documents · /audit-logs	Bearer + admin
Organisation	GET /api/org/members · POST /invite · PATCH /:id/role · DELETE /:id · GET /stats	Bearer + manager
Health	GET /api/health · GET /api/meta	public

Table 4. The 71-path API surface, grouped by module.

6.3 How the client consumes it

All HTTP traffic in the app flows through one module, `core/api/ApiClient.js`. It is a thin wrapper around fetch with four responsibilities. It prefixes every call with the configured API URL and attaches the stored access token. It unwraps the envelope, so presenters receive plain data or a typed ApiError carrying the status and code. It implements the refresh dance: on 401 TOKEN_EXPIRED it performs a single-flight refresh — concurrent failing calls share one refresh request — then retries the original call exactly once; on 409 TOKEN_REUSE it clears the session and returns the user to the login screen. And it centralises multipart construction for uploads, so the document-picker's file handle plus

metadata fields become one correctly framed form-data body. Presenters call the module-level endpoint builders, never raw URLs, which means the contract lives in exactly one file on the client too.

6.4 Sharing and verification flows from the API's point of view

Sharing is the module where the API design pays for itself. When an owner mints a share, the server generates a 43-character URL-safe token, hashes any passcode, stores the expiry and optional view limit, and returns the public URL. The public side is unauthenticated but never anonymous to the owner: GET the token returns only document metadata, and POST open runs the atomic guard UPDATE described in chapter 4 — if it moves one row, the view is counted and the owner is notified; if it moves none, the server re-reads the row to answer precisely why not: expired, revoked or exhausted. Downloadable shares add one more artefact: a single-use, five-minute ticket whose redemption is itself a guarded update, so a screenshot of a download URL dies in minutes. The newest capability is the OTP gate: an owner can mint a link that answers 401 SHARE_OTP_REQUIRED to everyone, email a six-digit code to exactly one designated address, and let open-with-otp exchange that single-use code (plus the passcode, if set) for one counted open — so even a forwarded link is worthless without the doorman's handshake. Verification requests follow a deliberate two-step flow — the owner mints and approves before the public link yields results — and the public verify endpoint answers with a proof payload (document title, type, owner name, SHA-256, issue and verification dates) and never with file bytes.

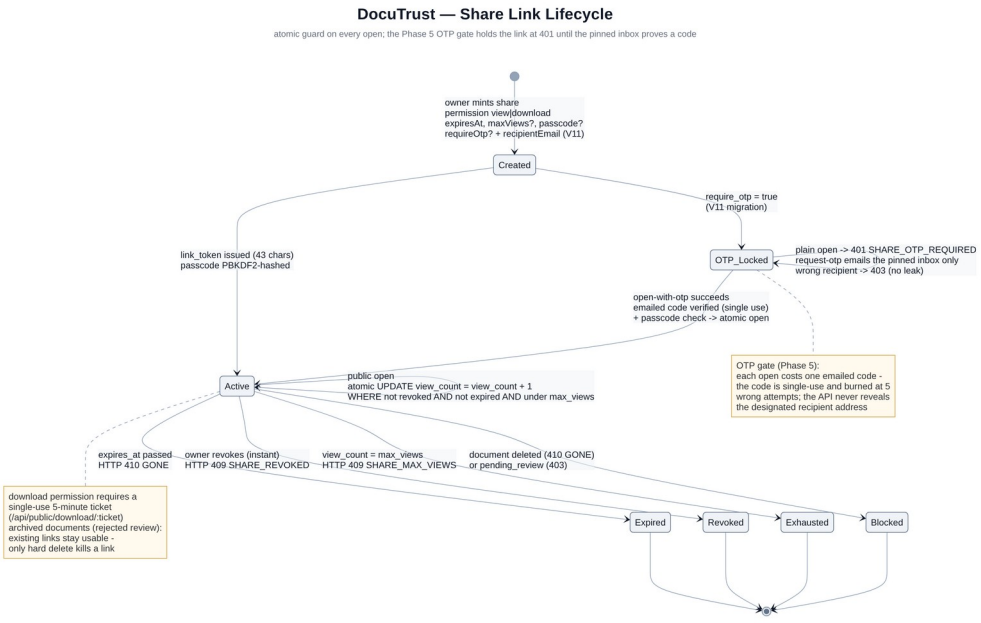


Figure 4. Share-link lifecycle and the atomic guard that enforces it.

6.5 SEO metadata

The brief requires metadata for SEO, and the natural home for it is the one part of the system a search engine can actually visit: the public verification page. When a browser (rather than the app) requests a verification token with an Accept header preferring HTML, VerifyService renders a small, fast, standards-compliant page carrying a title, a meta description, Open Graph tags for social previews and JSON-LD structured data describing the verification event. The same endpoint answers with JSON for the app, so one URL serves both audiences with content negotiation. On the client side, the Expo configuration carries the app's name, description and deep-link scheme, and the /api/meta endpoint exposes name, tagline, role list, screen and feature counts for the About screen — structured metadata all the way down.

7. MVP Pattern and Frontend Microservices

7.1 MVP in one sentence each

MVP stands for Model-View-Presenter, and it is the answer to a simple question: where does the logic live? The **View** is the React component — it draws what it is told and forwards taps. The **Model** is the data layer — the microservice's api.js and model.js functions that talk to the REST server. The **Presenter** is the decision-maker in between: it validates forms, decides what to fetch, holds a tiny state machine (idle, loading, success, error, empty) and calls the view back with instructions. The View never talks to the Model directly, which is the rule that makes the pattern worth its cost.

The practical payoff is calm screens and reviewable logic. Because a presenter is a plain JavaScript class with a constructor-injected API module, its decisions are plain to read and to exercise — a login with MFA pending routes to the MFA screen, a paginated list accumulates results across load-more calls, an error maps to a friendly message. Because every screen passes through the same BasePresenter lifecycle, every screen shows the same skeleton loaders, the same empty states and the same toast errors, which is why the app feels hand-made rather than assembled from forty-six one-off scripts. A single `usePresenter` hook wires a presenter's state into React's render cycle and detaches cleanly on unmount, so the pattern costs each screen about three lines of glue.

7.2 Frontend microservices

Microservices in the frontend do not mean nine little apps being deployed separately; they mean nine modules with hard walls between them. Each module under `src/microservices/` owns its API calls, its models, its presenters and nothing else; screens are dumb views registered in a central SCREEN_REGISTRY that maps forty-six names to components, modules and titles; navigation, deep links and notification taps all resolve through that registry. The result is that the sharing module can be rewritten without touching documents,

a new member can own one module end to end, and the defence question where does sharing logic live has a one-directory answer. The architecture mirrors the backend's modular monolith on purpose: same walls, same language of boundaries, same story.

8. The Six Hardest Challenges, Solved Twice

Every hard challenge in the brief is solved at two levels — once in the database and API design, and once in the code that runs per request. Presenting both layers is the strongest way to answer the inevitable so how does it actually work question. This chapter walks the six challenges in the order the brief lists them.

8.1 Access control

Authorisation is enforced three times, on purpose. Coarse role checks happen in middleware: an admin route wrapped in requireRole('admin') refuses members with 403 before any handler runs, and every refusal writes an audit row. Ownership checks happen in SQL: queries for a document are literally SELECT ... WHERE id = ? AND owner_id = ?, so user B asking for user A's document receives a 404, not a 403 — the system does not even confirm that the document exists. Finally, field-level shape is fixed by the schema: statuses and roles are CHECK-constrained, so an API bug could not write role='admin' into a row without the database refusing. The demo is one curl with a member token against /api/admin/users, and the audit log entry that appears beside it.

8.2 Private file delivery

Files live in a vault directory as AES-256-GCM ciphertext with a per-file random initialisation vector; the key comes from the environment, never from the repository. There is no public URL: downloads are GET endpoints behind the authentication middleware, the ownership WHERE clause, and a streaming decryptor that verifies the GCM tag and recomputes the SHA-256 on the way out. For public link shares, the download step requires a single-use ticket minted only when the guard UPDATE passes, valid five minutes, dead after one redemption. The net effect is that a leaked URL is worthless twice over: it points at ciphertext, and the endpoint it names will refuse anyone who has not just passed the guards.

8.3 Expiring links and revocation

Expiry and revocation share one mechanism: both are columns checked at request time, never at creation time. The atomic guard UPDATE encodes the whole policy — revoked_at IS NULL AND expires_at > now() AND (max_views IS NULL OR view_count < max_views) — inside a single statement, so even two simultaneous opens cannot race past an exhausted limit. Revocation is therefore instant: the owner taps Revoke, the row's revoked_at

is stamped, and the very next open anywhere in the world fails with 409. The edge-case script demonstrates this with a link whose lifetime is about thirty-six seconds: it works, then it does not, then the app shows exactly the right reason.

8.4 Audit trails

Two audit layers exist because they answer different questions. document_history answers what happened to this document — created, version added, shared, downloaded, status changed — and is fed automatically by a database trigger so no code path can skip it. audit_logs answers who did what to the platform — logins, failures, downloads, revocations, admin decisions, refused privilege escalations — with IP, user agent and a JSON detail column, and is written by the services on every security-relevant branch. Both tables are append-only by trigger: an UPDATE or DELETE raises an exception, and the defence demo shows the SQL editor throwing that exception live.

8.5 Document-version integrity

Every version row pins the SHA-256 of the exact encrypted bytes, and the document row mirrors the newest version's digest through a trigger. On every download the server recomputes the digest while streaming and compares; a mismatch means the stored object changed underneath the system, and the response aborts with an integrity failure plus an audit row. The verification feature exposes this property to the world: a proof link reports the document's stored hash, and if someone edits the vault file by hand — exactly what the tamper demo does — the next verification returns tampered instead of valid. Integrity is therefore not a claim but a checkable answer, which is the strongest form a claim can take.

8.6 The administrative review process

Some documents should not become active the moment they are uploaded, so DocuTrust routes medical and financial types into an admin_reviews row and sets the document's status to pending_review; a schema-level CHECK makes it impossible for such a document to be active with review_status not submitted. Admins work a queue joined against its targets, approve or reject with a note, and every decision updates the target, notifies the owner and writes an audit row. The same queue machinery handles suspicious accounts and flagged verification requests, which is why the brief calls it a process rather than a screen.

8.7 The OTP security layer (post-audit addition)

The security audit that followed the first build produced one structural conclusion: any secret an API response can carry, that response can leak. The old password-reset flow returned the reset code in the response body and the organisation invite returned a temporary password — convenient for demos, fatal as a pattern. The overhaul replaced both with one engine: a six-

digit one-time code, hashed with a purpose-and-address salt before storage, single-use, ten minutes from issue, dead after five wrong attempts, and delivered only by email through Brevo. Five flows run on it — password reset, the email MFA fallback at sign-in (the server answers 440 with a challenge token that the emailed code redeems for real session tokens), step-up confirmation for destructive actions like permanent deletion and password change (without a fresh emailed code the server answers 403 STEP_UP_REQUIRED), OTP-gated share links where the link stays locked at 401 until one designated inbox proves a code, and organisation invites that the newcomer claims with a self-chosen password instead of a temporary one.

9. How Correctness Is Shown

The project carries its proof in the open. The build is the first gate: `mvn clean package` must turn a clean clone into the runnable jar — Maven resolves the eight pinned libraries, and the pom's own comment declares that the build's job ends there. The boot is the second gate: fail-fast configuration checks print human explanations instead of letting a broken setup start, and the proof line `migrations=12, routes=74` is printed on every boot, so the schema and the menu can be counted live.

The third gate is behaviour, and it is demonstrated live: manual API calls (Postman/cURL) walk every flow — including the negative paths, where the honest refusals are the proof — and the emulator drives the same flows from the client side. The audit's finding IDs are greppable in the code comments, so any claim about a fix can be traced to the exact line in seconds.

Layer	How it is checked	What it proves
Build	mvn clean package	eight pinned libraries resolve; runnable jar packs
Configuration	fail-fast boot gates	weak secrets and bad key shapes are refused with the remedy printed
Schema	boot log + SQL editor	migrations=12; append-only triggers raise on edits
API behaviour	manual API calls (Postman/cURL)	every flow, including negative paths, live
Mobile	emulator (npx expo start)	46 screens driven against the live API

Table 5. How each layer is shown to work.

10. How to Run Everything

10.1 The database (Supabase)

Create a project on Supabase, open its SQL editor and either paste the eleven migration files in order or let the server's migration runner apply them on first boot. Copy the connection string from Supabase's settings — the pooler form, jdbc:postgresql://... — into the server's environment file. If your classroom has no reliable internet, the fallback is any local PostgreSQL instance; the schema is ordinary Postgres and knows nothing about Supabase specifically.

10.2 The server

From the server directory: copy .env.example to .env and fill in the database URL, user and password, a JWT secret and a vault key (the file comments show how to generate both with openssl). To send real OTP emails, set MAIL_MODE=live with your Brevo API key and sender address; the default MAIL_MODE=dev prints codes to the console and echoes them in an X-Dev-OTP header instead. Then build with mvn clean package and boot with mvn exec:java — Maven resolves the exact runtime jars itself — one command, nothing else required. On first boot you will see the migration runner apply V01 through V11, the seeder create the demo accounts, and the scheduler announce its sixty-second tick. Health check: GET /api/health should answer with database true. The demo accounts are admin@docutrust.app / Admin#2024, manager@docutrust.app / Manager#2024 and member@docutrust.app / Member#2024, plus student1 to student3 for role play.

10.3 The mobile app

From the mobile directory: npm install once, then npx expo start, and scan the QR code with the Expo Go app on a phone, or press i / a for the iOS or Android simulator. Point the app at the server by setting EXPO_PUBLIC_API_URL to the machine's LAN address in mobile/src/core/config/env.js. Sign in as the member account, upload a document, share it, revoke it, and watch the notifications arrive. Every flow can then be exercised live — manual API calls against the server perform the same steps the emulator performs.

Step	Command	Expected result
Database	apply migrations (or first server boot)	schema_migrations lists V01–V11
Server	mvn clean package, then mvn exec:java	health endpoint answers; seed users exist
Mobile	npm install && npx expo start	app opens on device/simulator
API flows	manual calls (Postman/cURL)	every flow answers, negative paths included
Mobile	npx expo start	app drives the same flows on the emulator

Table 6. Bring-up sequence.

11. The Three-Week Roadmap

The project was planned as three weekly sprints with a demo every Friday and one student owning each backend module together with its screens and tests — vertical slices rather than horizontal layers. Week one built the foundations: the repository and router skeleton, the first schema migrations, the cryptography services, the auth and document endpoints, and the first green end-to-end path from register to upload. Week two built product depth: all forty-six screens against the live API, the expiry scheduler, notifications, the admin review queue, and the first full pass over the two-hundred-feature register. Week three was hardening and rehearsal: the ten edge-case scripts, performance passes on search and pagination, a documentation freeze, individual two-minute pitches and a mock defence with hostile questions. The full week-by-week grid, including each student's task column and a risk register, is in docs/ROADMAP.md.

12. The Edge-Case Demonstrations

The final demo is scripted as ten reproducible edge cases, each with expected server answers; the full scripts live in docs/EDGE_CASES.md. Their common design is that each one makes a security claim falsifiable in front of the audience: a link that dies mid-demo proves expiry is checked at request time; a reused refresh token that nukes its own family proves reuse detection; a member token refused by the admin endpoint proves the role guard does not trust the UI; a member asking for someone else's document receiving a 404 proves ownership lives in SQL; two simultaneous version uploads proving the UNIQUE constraint; an oversized or mime-spoofed upload refused before touching the vault; and the tamper demo flipping a verification result from valid to tampered proves version integrity end to end. Ten minutes of these demonstrations answer more questions than an hour of slides.

13. Features, Roles and Screens at a Glance

The repository's docs/FEATURES.md itemises 216 core features with stable identifiers FT-001 to FT-216 across ten modules, plus the 18 post-audit security features FT-217 to FT-234 that document the post-audit hardening, from the OTP overhaul to the Active Sessions ledger, so every claim in this guide can be traced to a line in a table. The app's forty-six screens are registered in one file, mobile/src/screens/index.js, which is both the navigation source of truth and a convenient count for examiners. The three roles have clearly separated surfaces: members get the vault, folders, sharing, verification and notifications; org managers additionally get member management; admins get the dashboard, review

queue, user directory, platform documents and audit log browser. The register below summarises the module map; the defence guide (DEFENCE_GUIDE.md at the repository root) and the defence handbook map the same modules to the six team members.

Module	Features	Highlights
M1 Auth & access	FT-001–024	JWT + rotation, TOTP MFA, email-OTP fallback, step-up, rate limiting
M2 Documents & vault	FT-025–070	Encrypted upload, versions, search, trash, expiry radar
M3 Sharing & permissions	FT-071–096	Expiring links, view limits, passcodes, OTP-gated links, instant revoke, tickets
M4 Verification	FT-097–116	Proof links, tamper detection, SEO verify page
M5 Notifications & settings	FT-117–132	Scheduler-driven alerts, per-type preferences
M6 Administrative review	FT-133–156	Review queue, decisions, audit browser
M7 Organisation	FT-157–172	Members, invites, role changes, org stats
M8 Mobile experience	FT-173–192	46 screens, MVP, design system, deep links
M9 Server platform	FT-193–208	Router, middleware, pool, migrations, scheduler, seeder
M10 Data & integrity	FT-209–216	Constraints, triggers, views, trigram indexes, seeds
Post-audit security addendum	FT-217–234	email_otps engine, Brevo delivery, step-up, gated shares, the Active Sessions ledger

Table 7. The 232-feature register by module (216 core + 16 OTP security).

The *DocuTrust Defence Handbook* in the same folder holds seventy-nine likely examination questions with model answers, the per-student packs for all six members and a demo-day survival guide; chapter 14 that follows distils the eight core concepts once more for absolute beginners. The repository's docs folder holds the API contract, roadmap, edge-case scripts and diagram sources that this guide summarises.

14. Explain It Like I'm 5 — The Eight Core Concepts

This chapter assumes the reader has never written a line of code. Each of the eight sections explains one core concept with an everyday analogy, assigns it to the student who should rehearse it hardest, and ends with a one-sentence version you can say when the examiner asks for the short answer. Every folder name, number and library mentioned here is real — open the folder during the defence and point at it.

14.1 React Hooks — the sticky notes that remember things (rehearse: Student 4)

A phone app screen is redrawn hundreds of times a second — every tap, every new piece of data. But a drawing has no memory: wipe it and start again, and nothing is left over. A **hook** is a sticky note that React — the drawing library — keeps beside the drawing. The screen is wiped and redrawn all day long, but the sticky notes stay on the desk, remembering things between redraws. The word hook literally means that your screen component hooks into React's memory. Three sticky notes matter in our app.

useState — the sticky note with a magic pen. You write a value on the note, for example "unread notifications: 3", and the screen copies whatever is on the note. When a new notification arrives, you call the pen that came with the note; the note now reads 4, and React automatically redraws the screen. You never redraw anything by hand — changing the note is the whole job. Every counter, form field, loading flag and error message in our 46 screens lives on such a note.

useEffect — the "after you finish drawing, run this errand" slip. Drawing a screen should stay fast and dumb. The errands — fetching the document list from the server, starting a countdown, subscribing to notifications — go on the slip, and React runs each errand after the drawing is done. The slip also has a cleanup line: when the screen closes, the timer is stopped and the subscription cancelled, so nothing keeps running in the background like a tap left on.

usePresenter — our own custom sticky note, the one an examiner will ask about. DocuTrust does not let screens talk to the server directly; each screen gets a **presenter** — a plain-JavaScript "waiter" holding the screen's brain: what to fetch, what counts as a valid form, what to show while loading, on error, on empty. `usePresenter` is the three-line glue that hires the waiter when the screen opens, passes the waiter's status onto a sticky note so the screen redraws as it moves idle → loading → success → error, and fires the waiter when the screen closes so no presenter haunts a closed screen. One hook, and 46 screens all behave the same way.

One-sentence version: a hook is a sticky note that remembers things between redraws; useState remembers a value, useEffect remembers an errand to run after drawing, and our usePresenter remembers to hire — and later fire — the screen's waiter.

14.2 The Overall Architecture — the customer, the waiter, the kitchen (rehearse: Student 1)

DocuTrust is a restaurant with three rooms. The **customer** is the mobile app on your phone, built with React Native and Expo. It is good-looking and polite, but it has no kitchen: it cannot store your documents, check passwords or decide who may see what. It only orders.

The **waiter** is the backend server, a single Java program. Every wish from the customer — "show my documents", "share file X until Friday", "email me a login code" — is spoken to the waiter, never shouted into the kitchen. The waiter does four things, always in order: checks you are who you claim to be (a digital ID card called a JWT), checks you are allowed to order that (role and ownership rules), walks the order into the kitchen, and carries the plated result back on a standardized tray (a JSON envelope, explained in section 8). Our restaurant has exactly 74 dishes on the menu — the 74 registered API routes, a number printed when the server boots, so it is checkable rather than claimed.

The **kitchen and pantry** are the services and the PostgreSQL database. The chefs — the service code such as DocumentService, ShareService and OtpService — do the real cooking: encrypting files, minting share links, hashing one-time codes. The pantry is the database: fifteen filing drawers (tables) that remember every document, user, share link and audit entry, with a strict head clerk who refuses orders that break the house rules even when the chef made a mistake. Why three rooms? So each can be fixed alone: this sprint the entire OTP flow was rebuilt inside the kitchen, and the customer barely noticed — only the ordering script changed.

One-sentence version: the app is the customer, the server is the waiter with 74 things on the menu, the database is the pantry — and no room ever walks into another room uninvited.

14.3 The Frontend Folders — it's like a smartphone (rehearse: Student 4)

Open mobile/src/ and you see seven folders. The easiest way to explain them: the app is a smartphone, and each folder is a part of the phone.

`screens/` are the different **apps you open** — forty-six of them: a Login app, a Documents app, a Trash app, a Share app. Each screen is one full-page view, and they are deliberately dumb: they draw what they are told and forward taps, like an app that only shows what its data says. `components/` are the **buttons on the screen** — small reusable pieces (a document card, a password input, a toast pop-up, a loading skeleton) drawn once and used on many screens, which is why the whole phone feels consistent.

`microservices/` are the **invisible background processes that fetch your data** — nine folders (auth, documents, folders, sharing, verification, notifications, org, profile, admin), one per feature area. Each holds that domain's api files (how to ask the waiter), model files (what the data looks like) and presenter files (the screen's brain). The user never sees these; like background processes they do the fetching and deciding. Why separated? So six students

could work at once without stepping on each other — the sharing student rebuilt her whole OTP-gating flow and never touched the documents folder. The remaining folders are small: `navigation/` is the home-screen gesture (which app opens when you tap what), `mvp/` is the house rulebook every screen's waiter follows, and `core/` is the phone's settings — the server address and the design tokens.

One-sentence version: screens are the apps you open, components are the buttons on them, microservices are the invisible background processes fetching the data — and navigation is the home gesture that moves between apps.

14.4 The Backend Folders — a well-organized restaurant (rehearse: Student 1)

Open server/src/main/java/com/docutrust/ and you see the restaurant's staff chart. The `controller` folder is the **waiters taking orders** — ten of them (AuthController, DocumentController, ShareController and so on), one per section of the menu. A waiter's job is narrow on purpose: receive the order, check the ID, hand it to the kitchen, carry the plate back. Waiters never cook.

The `service` folder is the **chefs cooking** — the real logic: hashing one-time codes, encrypting files, enforcing share rules. Chefs never talk to the customer directly; that is the waiter's job. This split is why the team could swap the recipe for password resets — from "send a temporary password" to "send a one-time code" — without changing a single waiter. The `db` folder is the **pantry and the head clerk**: Database (the walk-in cooler connection), Migrator (the eleven numbered instruction sheets that build the pantry from scratch), Rows and Tx (the rules for reading a shelf and for all-or-nothing orders). The `vault` folder is the restaurant safe — files sit there encrypted with AES-256-GCM, a standard strong enough for banks.

The `auth` folder is the **ID checkers at the door**: PasswordService (checks your password without ever knowing it — it only compares scrambled fingerprints), TokenService (prints and re-prints your session wristband), MfaService (the second-proof checker) and RateLimiter (the bouncer who says "too many tries, wait a minute"). The `http` folder is the **uniforms and order pads** — Router (the 74-item menu index), Middleware (the security chain every order passes), Ctx (the standard order-slip format). Around them: config is the manager's opening checklist (it refuses to open with weak keys), infra is the office dashboard showing pool and traffic statistics, model holds the standard plate shapes, and util is the toolbox. Why separated? A new chef can be hired without retraining the waiters — and during the OTP overhaul the team rewrote chefs (OtpService, MailService) while the waiters and the pantry never changed.

One-sentence version: controllers are the waiters taking orders, services are the chefs cooking, db is the pantry, auth checks IDs at the door — and no waiter ever walks into the pantry himself.

14.5 The Database — a highly organized filing cabinet (rehearse: Student 1)

A **relational database** is a filing cabinet with strict rules, and PostgreSQL — the one we chose — is the strictest, most trusted cabinet on the market. The cabinet has fifteen drawers (**tables**): users, documents, document_versions, shares, email_otps, audit_logs, and so on. (Peek with a database tool and you will see a sixteenth — schema_migrations — but that is not one of ours: it is the moving truck's own clipboard, where the migration runner writes down which numbered instruction sheets it already delivered.) Inside each drawer are **rows** — index cards, one per user, per document, per share link. Each card has the same printed **columns**: a user card always has name, email, role, created-at. You never find a hand-scribbled card with a different shape — the drawer's form is fixed when the drawer is built.

A **foreign key** is a cross-reference written on the card. A document card says "owner: see card 42 in the USERS drawer." The clerk refuses to file a document whose owner card does not exist, and — by our setting — refuses to destroy a user card while documents still point at it. That is what "relational" means: the cabinet itself guarantees the references make sense, so a typo or a bug cannot create an orphan.

Why PostgreSQL, in plain words? Four reasons. First, it **enforces rules**: a CHECK constraint is a house rule the clerk applies even to the boss — a medical document physically cannot become "active" without review, no matter what buggy code asks for. Second, it **keeps honest history**: triggers make the audit cards append-only — they cannot be edited or torn out, only added. Third, it is **free and battle-tested for over twenty-five years**, running everywhere — including Supabase, a free cloud landlord for it, so our project costs nothing to host. Fourth, it speaks a standard language, SQL, that all of us could write. The cabinet was assembled from **eleven numbered instruction sheets (migrations)**: V01 builds the users drawer, V10 adds the email-OTP drawer, V11 adds three fields to shares — so any team member can rebuild the identical cabinet from nothing, in order, in minutes.

One-sentence version: tables are drawers, rows are index cards, foreign keys are cross-references the clerk refuses to break — and PostgreSQL is the strict, free, 25-year-old cabinet that enforces our house rules even against bugs.

14.6 The Server — a 24/7 receptionist answering phones (rehearse: Student 6)

What is a web server? It is a program that never sleeps, sitting by a phone that never stops ringing. Ours is one single Java process, and its day looks like this. The phone rings — a

request arrives from the app. The receptionist picks up **within milliseconds, always**, whether it is one call or fifty at once. It asks "who is calling?" and checks the caller's ID card (the JWT): no card, or a fake card, gets a polite refusal — status 401. It looks up the requested department in its directory of **74 extensions** (the router) — and anything unknown gets a clean "no such extension" (404), never a crash. Then it walks the request to that department and carries the answer back, always on the same standardized tray (the JSON envelope), with a rubber stamp saying how it went.

Three more duties make ours a good receptionist. It **writes down every call** — one log line per request (what was asked, when, how long it took), and every security-relevant event into an append-only audit book that even the boss cannot tear pages from. It **limits repeated callers** — someone asking for a password reset eleven times in a minute is told to wait (rate limiting), which is how the system shrugs off people who try to guess codes. And it **refuses to open the office with broken locks** — at start-up it checks its own keys and configuration, and if the signing key is weak it refuses to start and prints exactly what to fix. That is called fail-fast: a two-minute fix on the ground beats a mystery crash in the air.

One-sentence version: a web server is a 24/7 receptionist: answers every call instantly, checks IDs, forwards to one of 74 departments, writes down everything, and never hangs up rudely.

14.7 The Tech Stack — what we chose and why (rehearse: Student 6)

Every technology in the project earns its place in one of three ways: the brief told us to, it is the boring trusted option, or it is the standard library for a problem too dangerous to hand-roll. The table below is the whole list — learn the "why" column and the examiner's question is already answered.

Java 21 is the language of the server — one recipe book that runs on any machine with a Java reader installed. The brief forbids frameworks, and Java is a twenty-five-year-old workhorse: strict (mistakes are caught before running), fast, and familiar to all of us; version 21 is a long-term-support release, the sturdy supported-for-years edition. **PostgreSQL, hosted free on Supabase**, is the filing cabinet of section 5 — strictest rule enforcement, free hosting, decades of trust. **React Native 0.88 with Expo SDK 58** is the phone-app builder: you write the app once in JavaScript and it runs on both iPhone and Android, like a translator fluent in both — and Expo is the training wheels: scan a QR code and the app runs on a real phone immediately, no app store needed for demos. The app even has its own mechanic: `npx expo-doctor` inspects the dependency list and currently reports 20/20 checks clean, and `npx expo install --fix` keeps every library on the version the SDK certified. Where the app finds the server is configuration, not code: `mobile/.env` holds EXPO_PUBLIC_API_URL and

`src/core/config/env.js` reads it, while a root `.gitignore` keeps the real .env and local build output out of GitHub.

Brevo is the postman — an email-delivery service with a simple API. One-time codes must reach real inboxes; Brevo has a free tier and a two-line sending call, and our MAIL_MODE switch lets us rehearse the whole OTP flow without emailing anyone. **HikariCP** is the restaurant host for the database: it keeps a fixed set of tables (connections) ready and seats each query instantly — hand-rolling that seating is a classic source of 3am crashes. **Bucket4j** is the ration-book stamp that counts requests per minute per user, with budgets tuned per endpoint scope (20 a minute for auth, 30 for public share pages). **Caffeine** is the noticeboard by the kettle — the few hot facts everyone re-reads, kept in memory with size and time limits so the board can never go stale. **Dropwizard Metrics and Sentry Spotlight** are the office dashboard and the CCTV timeline: /api/health shows pool and request statistics live, and Spotlight (optional, one flag) draws the full timeline of a request across the system.

One-sentence version: every choice is either "the brief told us" (pure Java, no framework), "the boring, trusted option" (PostgreSQL, Java 21 LTS), or "the standard library for a problem too dangerous to hand-roll" (HikariCP, Bucket4j) — and we can name the danger for each.

14.8 The Data Consumption API — the standardized order form (rehearse: Student 3)

How does the app actually talk to the waiter? Two conventions make it work, and both are agreed paperwork. **HTTP methods are the verbs on the order pad**. Every request says what kind of wish it is. **GET** means "just let me look" — opening the documents list changes nothing. **POST** means "here is something new" — uploading a document, registering, requesting an OTP code. **PUT** means "replace this whole thing" — overwriting the notification settings. **PATCH** means "amend one field" — renaming a document. **DELETE** means "take this away" — moving a document to trash. The same address can behave differently by verb: GET /api/documents lists, POST /api/documents uploads, DELETE /api/documents/12 trashes document 12. Reading is safe; writing is checked twice — ID first, then permission.

JSON is the standardized order form — a way of writing data that both the app and the server read: labelled boxes in fixed positions, like {"title": "Passport", "status": "active"}. The phone speaks Android and the server speaks Java, but neither cares: they only swap forms. Every answer arrives in the **same envelope**: ok true with the data inside on success; ok false with an error code and a friendly message on failure — so the app opens every envelope the same way, and a screen that hits an error always finds a code it can translate into plain words.

The rubber stamp on the envelope is the **status code**, three numbers with fixed meanings worth memorising: 200 "here you go"; 400 "your form was filled wrong"; 401 "ID card missing or fake"; 403 "real ID, but not allowed" — or, with STEP_UP_REQUIRED, "allowed, but prove a second time first"; 404 "no such thing — and we will not say whether it exists for someone else"; 409 "conflict — the link is expired, revoked or out of views, or that token was reused"; 429 "too many requests, wait a minute"; 440 — our own stamp — "second factor needed, check your email"; 500 "our mistake, sorry — the details are in the log, not in your face."

One-sentence version: the frontend hands the backend standardized order forms — five verbs on the pad, JSON as the form's printed boxes, one envelope for every answer, and a rubber stamp (the status code) saying exactly how it went.

These eight explainers are also in the repository's *DEFENCE_GUIDE.md* (Part 2), next to the six per-student defence packs — the two documents share the same analogies on purpose, so the team tells one story.

15. The Six Defence Domains in Plain Words

Six students, six domains, one story. This chapter distils each Masterclass file in `docs/defence/` into the three things a non-technologist needs: what the person owns in one sentence, the analogies you can reuse in any explanation, and a few glossary entries translated into everyday life. The full deep dives — file-by-file ownership maps, code walkthroughs and the personal Q&A banks — live in the Defence Handbook and in the six files themselves.

> *Note: every claim in this handbook is verifiable from the public repository alone — finding IDs are greppable in the code comments, mvn clean package builds the runnable jar from a clean clone, and every flow is demonstrated live at the defence with manual API calls (Postman/cURL) or on the emulator.*

Student 1 — Infrastructure & Database

In one line: The skeleton and the truth layer — the pure-Java server platform every request travels through, and the PostgreSQL schema whose invariants live in the database engine itself.

Analogies you can reuse

1. **HikariCP is a restaurant host.** Guests (database queries) don't wait for a free table to be built; a host keeps a fixed set of tables ready and seats the next guest at whichever frees up first. Before HikariCP we hand-wrote that seating logic; now it's a battle-tested library with the same seats (our code's API didn't change at all).

2. **Migrations are LEGO instructions in order.** Each numbered file (V01–V12) adds a few bricks — a table here, an index there — and the database is just the sum of the steps. Any team member can rebuild the exact same database from scratch, which is why our ER diagram and our production schema can never drift apart.
3. **The router is a hotel receptionist.** You say "I'm here for room 12" (`/api/documents/:id`) and the receptionist knows static room numbers take priority over "the room with whatever name" — exact matches win over wildcards. One receptionist, 74 known destinations, and anything unknown gets a clean 404 instead of a crash.
4. **The middleware chain is airport security.** Every passenger walks the same lane before reaching the shops: the passport scan is JWT authentication, the boarding-class check is the role guard, and the queue counter that says "too many people, wait" is the rate limiter. No shop — no controller — is ever reached without passing through the lane first, and the lane's order never changes, which is exactly why it is owned by one person.
5. **The append-only audit trigger is a cash-register till tape.** The tape only prints new lines; you cannot edit or un-print a line, not even the shop owner can, and at the end of the day the tape is the one thing everyone trusts in a dispute. Our `audit_logs` and `document_history` tables work the same way: the database itself raises an exception if anyone — application, administrator, or a hand-typed SQL command — tries to alter or tear out a line.

Glossary picks, translated

Term	Plain meaning, with an everyday analogy
Middleware	Airport security: the same lane every passenger walks through (ID check, boarding class, queue counter) before reaching any shop. In code: Middleware.authenticate, requireRole, rateLimited.
JWT (JSON Web Token)	A tamper-proof visitor badge: a signed card that states who you are and until when — the signature makes forging it as futile as photocopying a hologram. Ours is HMAC-SHA256, hand-built in JwtService.
HTTP status code	The rubber stamp on the reply envelope: 200 "here you go", 401 "no or fake ID", 403 "real ID, not allowed", 404 "no such thing", 429 "too many requests".
CORS	The guest list at the club door for web browsers: a header the server sends declaring which origins may talk to it, so a stranger's website cannot quietly use your logged-in session.

Term	Plain meaning, with an everyday analogy
Connection pool (HikariCP)	The restaurant host: a fixed set of ready "tables" (open database connections) that guests (queries) are seated at instantly, instead of building a new table per guest. HikariCP is the battle-tested professional host we hired to replace our hand-written one — same seats, and it reports room occupancy (poolIdle/poolActive).

Student 2 — Auth & OTP Security

In one line: how a stranger becomes a trusted user — and how a stolen password alone still is not enough.

Analogies you can reuse

1. **An OTP is a VIP wristband, not a key.**
2. **Step-up authentication is a bank vault inside a bank.**
3. **Refresh-token rotation is a tear-off ticket pad.**
4. **PBKDF2 password hashing is a one-way blender run 120,000 times.**
5. **Purpose-salting an OTP digest is stamping the key with its door.**

Glossary picks, translated

Term	Plain meaning, with an everyday analogy
OTP (one-time passcode)	A VIP wristband: handed to you at the door, works once, worthless tomorrow. Six digits, emailed, dead in 10 minutes.
PBKDF2 + salt	A recipe that stirs the password soup 120,000 times before serving — and every user's soup gets its own unique pinch of spice (the salt), so identical passwords still store differently and pre-computed "rainbow" recipe books become useless.
SHA-256 digest	A fingerprint of a message: any change in the input changes the fingerprint, and you cannot rebuild the message from it. We store fingerprints, never the secrets themselves.
Purpose-salted digest	The fingerprint taken over purpose:email:code instead of just the code — a key stamped with which door it fits, so a reset code can never open the share lock.
JWT (JSON Web Token)	A wax-sealed ID card: the server writes your details on it, stamps the seal with its secret key, and anyone can read the card but only the server can forge the seal.

Student 3 — Business Logic (Documents, Sharing, Verification)

In one line: the bytes and the doors — encrypted storage, versioned documents, and the share engine that faces strangers.

Analogies you can reuse

1. **The share guard is a nightclub bouncer with a clicker.**
2. **An OTP-gated share is a registered letter with a doorman.**
3. **Version pinning is a notarized photocopy.**
4. **The vault is airport luggage wrap with a built-in seal checker.**
5. **A download ticket is a cinema stub torn at the door.**

Glossary picks, translated

Term	Plain meaning, with an everyday analogy
AES-256-GCM	The airport luggage wrap for files: strong modern encryption *plus* a built-in seal, so the wrapped thing cannot be read or silently altered. Banks use the same family of standards.
GCM tag (128-bit)	The tamper-evident seal printed on the wrap: any modification of the stored ciphertext — even one flipped bit — makes the seal fail at unwrap time, and we answer "tampered" instead of serving bytes.
IV (initialisation vector)	The unique twist pattern given to each wrap: 12 fresh random bytes per file, written in front of the ciphertext, so two identical documents still encrypt to completely different bytes.
SHA-256 fingerprint	The notary's fingerprint of the exact bytes, taken while they stream past; store the fingerprint, and every future reader can check the file still matches.
Storage key	The safe-deposit box number (ab/<uuid>.bin), never the document's name — the filesystem reveals nothing about what is inside.

Student 4 — Mobile App

Analogies you can reuse

1. **The MVP pattern is a restaurant waiter.** The screen (view) is the dining room — it never walks into the kitchen (the API). The presenter is the waiter who takes your order, talks to the kitchen, and brings back the plated result; the model is the menu's ingredient list. When the kitchen changes a recipe, the dining room doesn't move — which is exactly how we swapped the whole auth flow for OTPs without rewriting a single screen layout.

2. **Single-flight token refresh is one umbrella in a sudden downpour.** Five people walk out at once, it starts raining — they don't each buy an umbrella; one person buys it and the rest share it. When five API calls all get 'token expired' at the same moment, one refresh request goes out and the other four wait for its result, instead of stampeding the server with four identical refreshes.
3. **The step-up UI is the "are you sure?" that actually checks something.** Normal confirmation dialogs are theater — everyone clicks OK. Ours can't be skipped: the app asks for a code that was just emailed, the server won't accept the destructive request without it, and the dialog you clicked is only the first of two independent proofs.
4. **The screen registry is the phone's home-screen gesture.** You don't tell the phone "open the fourth icon in the second row of the third folder" — you say the app's name, and the phone itself knows where it lives. Every navigation in our app — a presenter's `go()`, a notification tap, a `docutrust://` deep link — says the *name* from `SCREEN_REGISTRY`, and the registry alone knows which component, which module and which title that name means. Change where a screen lives once, in one file, and forty-five other screens never notice.
5. **AsyncStorage is the app's coat pocket.** When you leave a building you keep your key in your pocket, not in your head — next morning you reach for it and you're still you. When the app is killed, the session tokens live in the phone's keystore pocket (`expo-secure-store`), while the profile card that greets you stays in the AsyncStorage pocket (`docutrust.session.v1`), so reopening the app doesn't mean signing in again. And just as you'd empty the pocket of a stolen coat, `logout()` wipes the pocket and the server revokes what was in it.

Glossary picks, translated

Term	Plain meaning, with an everyday analogy
MVP pattern	Model-View-Presenter: the screen draws, the presenter decides, the model describes the data. Like a restaurant: the dining room (view), the waiter (presenter), the menu's ingredient list (model) — the dining room never walks into the kitchen.
Presenter	The screen's brain: a plain JavaScript class holding its state machine and its API calls, so the logic lives in one reviewable place without drawing anything. The waiter you always get, whatever table you sit at.
View / screen	The dumb drawing: it renders presenter state and forwards taps. The dining room that only displays plates.
usePresenter	The hook (React's sticky note) that connects one brain to one drawing: create it once, mirror its state, clean up when the screen closes. Hiring the waiter when you sit down and thanking them when you leave.

Term	Plain meaning, with an everyday analogy
BasePresenter state machine	Five statuses — idle, loading, success, error, empty — every screen passes through. The restaurant's standard placemats: waiting, serving, sorry-we're-out, the bill.

Student 5 — The 8-Pass Security Audit

Analogies you can reuse

1. **The 8-pass audit is a building inspection with 8 specialists.** One inspector checks the locks, one the wiring, one the fire exits... A general handyman catches obvious things; a specialist catches the gas leak behind the correctly-painted wall. Each pass had one theme and a checklist, so 'we looked at everything' meant something checkable.
2. **B-1 (fail-open revocation) is a burglar alarm that unplugs itself when it rings.** The old code detected a stolen token and tried to revoke the family — but it did the revoking inside the very transaction that then threw the alarm's exception. Both rolled back together: the alarm rang and un-rang in the same instant. The fix rings the alarm *after* the bolt has been thrown — revoke in one committed transaction, then raise the error.
3. **A greppable finding ID is the label on the rewired wire.** An electrician who rewires your house and says 'trust me' is not done; the junction box gets a written label saying what was wrong and what was changed. Every fix in this codebase carries its label — the finding ID in a comment — so any reviewer can trace what broke and why the fix is shaped the way it is.
4. **A finding ID is a named scar with a story.** "There was a bug once" heals into nothing; "B-1" is a scar you can point at — the skin remembers, and anyone can press it later and hear the story. Naming findings the way hospitals name procedures means the fix, the demo and the comment all use the same word, five months later, in three different files.
5. **Prioritizing the fixes is a hospital triage nurse.** Eight passes produce a waiting room full of problems; the nurse does not treat whoever arrived first, she treats the most contagious and most life-threatening first. Exploitability is how contagious a bug is — how easily a stranger can trigger it — and blast radius is how much of the system it infects. Fail-open token revocation is a haemorrhage; a mis-split migration comment is a splinter; and the splinter genuinely waits its turn.

Glossary picks, translated

Term	Plain meaning, with an everyday analogy
Finding ID	A stable name for one discovered weakness (letter = pass, number = order in that pass). A named scar with a story — everyone on the team says "B-1" and means the same wound.
Fail-open	A control that lets everything through *when it fails*. A doorman who faints and props the door open. Fail-closed is the opposite: he faints and the door locks.
Transaction / commit / rollback	An all-or-nothing bundle of database changes — every item scans or the whole basket goes back. *Commit* is the till drawer closing (changes become permanent); *rollback* is the receipt torn up and the shelf restocked, which is what an exception triggers.
Race condition	Two operations interleaving badly because both "checked" before either "acted". Two people reading the last concert ticket on the board and both walking to the counter.
FOR UPDATE	A database row lock: "I'm being served about this row — everyone else waits." The numbered ticket dispenser at the deli counter.

Student 6 — Deployment & DevOps

In one line: Getting it running — anywhere, in one command — and keeping it observable, repeatable and secret-clean.

Analogies you can reuse

1. **Fail-fast configuration is the pre-flight checklist.** Pilots don't discover a flat tyre at cruise altitude. Our server refuses to *start* with a weak signing key or a missing vault key, and prints exactly what to run to fix it — a two-minute fix on the ground beats a mystery crash in the air.
2. **MAIL_MODE is the flight simulator switch.** In `dev` mode the OTP codes appear in the console and the response header — you can rehearse the whole two-factor flow without emailing anyone. Flip to `live` and the same code path goes out through Brevo's real email service. Same cockpit, zero risk of spamming strangers while you learn.
3. **The migration runner is the moving truck with a checklist.** It looks at which numbered boxes (V01–V12) are already in the new house, carries in exactly the missing ones, in order, once each. Fresh database or five-months-old database, one boot command brings both to the same final layout.

4. **The health and metrics endpoints are the car's dashboard.** While driving you glance at the fuel gauge and the engine light instead of stopping and opening the bonnet every hour: `/api/health` shows the engine light (`db` up or down), the odometer (`requestsTotal`), and the average speed (`meanResponseMs`). The one-line-per-request log is the journey diary; the dashboard is how you notice a problem before the passenger does.
5. **`.gitignore` is the customs list at the border.** GitHub is a foreign country every file must legally enter, and the list says what may never cross: real keys and passports (`.env`, `*.key`), unpackaged luggage (build output and downloaded dependencies), and personal belongings (editor settings). Only the blank application forms are waved through (`.env.example`, `mobile/.env.example`) — so even a careless `git add .` smuggles nothing in.

Glossary picks, translated

Term	Plain meaning, with an everyday analogy
Environment variable / .env file	A note pinned to the process's noticeboard before it starts: the same program reads different notes on your laptop, my laptop and the examiners' — code unchanged. The .env file is the physical noticeboard, one KEY=value per line in the project folder, loaded at boot — the house where the real keys live, and which never travels.
.env.example	The blank application form: every variable documented with a placeholder, safe to publish, and — in our case — with placeholders the server actively refuses to run on.
Fail-fast	The pre-flight checklist: refuse to start with a known-broken setup and say exactly why, instead of taking off and crashing at altitude (E-3 and the other Config gates).
JDK / JRE	The kitchen versus the dining room: a JDK can *cook* (compile) Java, a JRE can only *serve* (run) it — which is why the pom declares maven.compiler.release 21: a JDK 21 builds the jar, any JRE 21 runs it.
Jar	A zip of compiled Java with a label saying where the front door (Main-Class) is — our server is one jar plus eight library jars.